综合实践报告

课程:机器学习	实验名称: 逻辑回归及应用	日期:3.25
姓名：雷文豪	学号:138022024031	专业班级: 人工智能

1 实验目的

掌握处理混合型数据（包括数值型和类别型）的方式，并加深对逻辑回归模型的理解。

2 实验内容提要与实验设备

1. 针对待解问题设计适合的求解方法。
2. 对给定数据集，设计求解过程、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 德国信用风险分析

✓ 填写要求：程序输出数字文字时只截图有数字文字的部分（小截图）；若插入文档的截图仍比较大，将截图拉小点，再进行一次截图；代码使用无修饰、单倍间距、黑色新罗马体。

3.1 任务概述

利用给定的**德国信用风险数据集**（`german_credit_data.csv`），对用户的信用等级进行分类（`Good` 和 `Bad`）。该数据集包含 1000 条记录，每条代表一个接受银行信贷的人。数据集经过特殊处理，只保留 10 个关键属性（除第一个 ID 外），每个人根据其属性集被划分为信用风险好（`Good`）或坏（`Bad`）。每个属性的说明如下所示（注意： NA 表示该数据缺失）：

- Age：年龄
- Sex：性别（男性，女性）

- Job：工作（0-非技术人员和非居民， 1-非技术人员和居民， 2-熟练，3-高技能）
- Housing：住房（拥有，出租，免费）
- Saving account：储蓄帐户（小，中，丰富，相当丰富）
- Checking account：支票帐户（小，中，丰富）
- Credit amount：贷方金额
- Duration：持续时间（以月为单位）
- Purpose：目的（汽车，家具/设备，广播/电视，家用电器，维修，教育，商业，度假/其他）
- Risk：风险等级（好，坏）

3.2 解答步骤

步骤一. 对数据进行统计分析，包括观察数据、对数据进行清洗和转换等。

步骤二. 不使用 `sklearn` 库函数，用 `NumPy` 编写 `Sigmoid` 函数：

$$g(\mathrm{z}) = \frac{1}{1+e^{-\mathrm{z}}}.$$

并且编写梯度下降法的更新程序。

步骤三. 性能评价的求解设计，本次实验暂时只使用分类错误率（Err）或分类正确率（Acc），如下所示：

$$\mathrm{Err}(f;\mathcal{D}) = \frac{1}{\mathrm{N}}\sum_{n=1}^{\mathrm{N}} \mathbb{I}(f(\mathbf{x}_n) \neq y_n),$$

$$\mathrm{Acc}(f;D) = 1 \_ \mathrm{Err}(f;D),$$

其中 f 是所设计的模型； D 表示数据集； $\mathrm{I}(\cdot)$是指示函数，当满足条件时等于 1，否则为0。

步骤四. 选择你认为最相关的两个特征，绘制分类决策边界线。

1.数据分析与预处理
（1）源代码：

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

# 读取数据
df = pd.read_csv("german_credit_data.csv")
# 删除多余列
df = df.drop(columns=["Unnamed: 0"], errors="ignore")

# 数据集概览
print("数据形状：", df.shape)
print("\n缺失值统计：", df.isnull().sum())

# 因变量二值化
df["Risk"] = df["Risk"].map({"bad":1, "good":0})
```

```
# 缺失值处理
df["Saving accounts"] = df["Saving accounts"].fillna("none")
df["Checking account"] = df["Checking account"].fillna("none")

# 验证缺失值处理结果
print("\n缺失值处理后统计：")
print(df.isnull().sum())

# 分类型特征编码
# 二分类无序特征
df["Sex"] = df["Sex"].map({"female":0, "male":1})

# 有序多分类特征使用有序整数编码
saving_map = {"none":0, "little":1, "moderate":2, "quite rich":3, "rich":4}
check_map = {"none":0, "little":1, "moderate":2, "rich":3}
df["Saving accounts"] = df["Saving accounts"].map(saving_map)
df["Checking account"] = df["Checking account"].map(check_map)

# 无序多分类特征使用独热编码
df = pd.get_dummies(df, columns=["Housing", "Purpose"], drop_first=True, dtype=int)

# 查看特征列
print("\n编码后所有特征列：")
print(df.columns.tolist())

# 数值型特征标准化
std_cols = ["Age", "Credit amount", "Duration"]
std_params = {}
for col in std_cols:
    mean_val = df[col].mean()
    std_val = df[col].std()
    std_params[col] = {"mean": mean_val, "std":std_val}
    df[col] = (df[col] - mean_val) / std_val

# 异常值处理
iqr_cols = std_cols
for col in iqr_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df[col] = df[col].clip(lower=lower_bound, upper=upper_bound)

# 特征工程
# 单位期限内的还款金额反映还款压力
df["Credit_Per_Duration"] = df["Credit amount"] / df["Duration"]
# 年龄与工作等级反映收入稳定性
df["Age_Per_Job"] = df["Age"] / (df["Job"] + 1)  # +1避免异常
# 信贷规模与年龄匹配度
df["Credit_Per_Age"] = df["Credit amount"] / df["Age"]

# 处理后的数据集概览
print("\n预处理完成后数据概览：")
print(df.info)

# 保存处理后的数据
df.to_csv("german_credit_processed.csv", index=False, encoding="utf-8")
print("处理后的数据集已保存到german_credit_processed.csv")

# 可视化部分
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
```

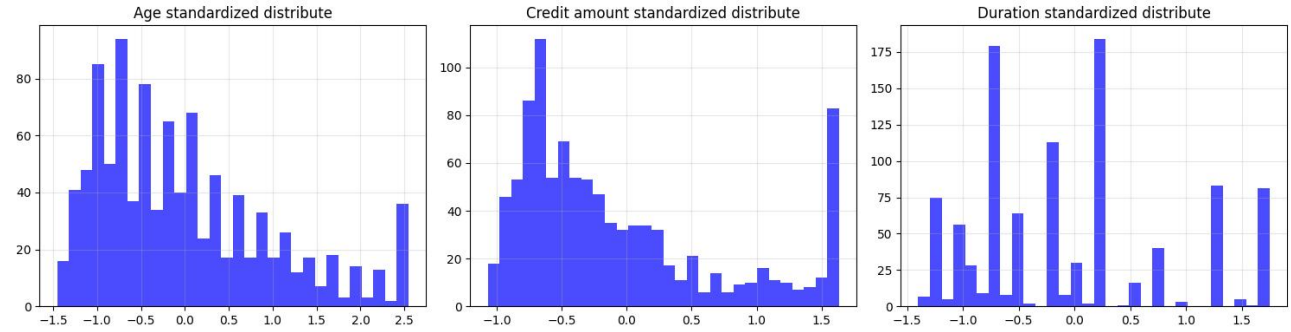
```
for idx, col in enumerate(std_cols):
    axes[idx].hist(df[col], bins=30, color="blue", alpha=0.7)
    axes[idx].set_title(f"{col} standardized distribute")
    axes[idx].grid(alpha=0.3)
plt.tight_layout()
plt.show()

# 绘制因变量分布
plt.figure(figsize=(6, 4))
df["Risk"].value_counts().plot(kind="bar", color=["blue", "red"])
plt.title("credit level distribute")
plt.xticks(rotation=0)
plt.grid(alpha=0.3)
plt.show()
```

（2）结果：

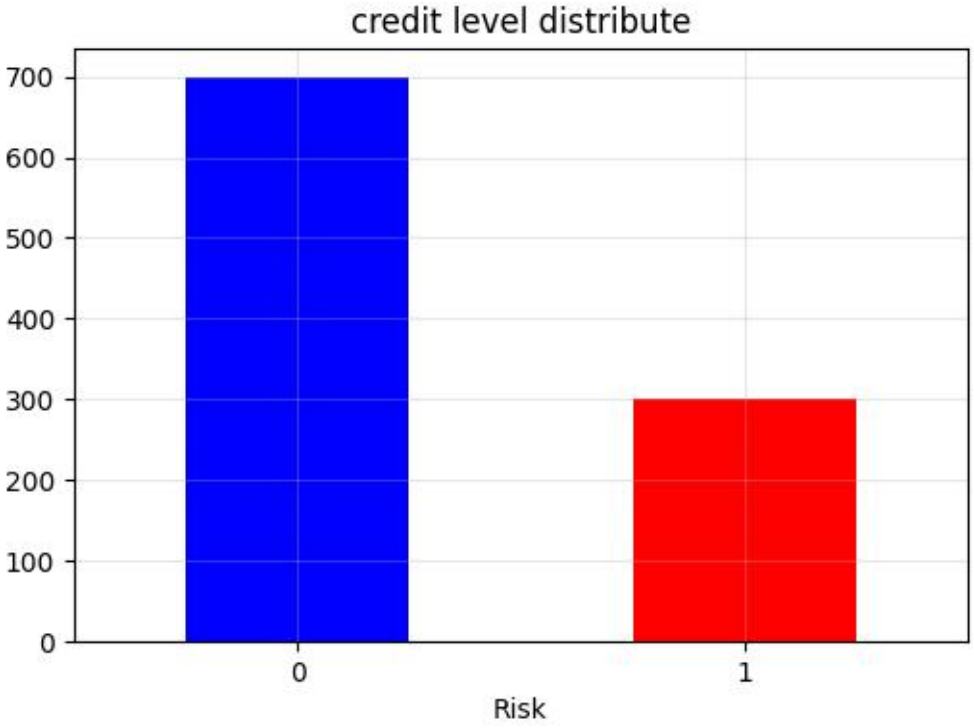
```
数据形状： (1000, 10)

缺失值统计： Age                 0
Sex                 0
Job                 0
Housing             0
Saving accounts     183
Checking account    394
Credit amount       0
Duration            0
Purpose             0
Risk                0
dtype: int64

缺失值处理后统计：
Age                 0
Sex                 0
Job                 0
Housing             0
Saving accounts     0
Checking account    0
Credit amount       0
Duration            0
Purpose             0
Risk                0
dtype: int64

编码后所有特征列：
['Age', 'Sex', 'Job', 'Saving accounts', 'Checking account', 'Credit amount', 'Duration', 'Ris
k', 'Housing_own', 'Housing_rent', 'Purpose_car', 'Purpose_domestic appliances', 'Purpose_educ
ation', 'Purpose_furniture/equipment', 'Purpose_radio/TV', 'Purpose_repairs', 'Purpose_vacatio
n/others']

预处理完成后数据概览：
<bound method DataFrame.info of          Age  Sex  Job  ...  Credit_Per_Duration  Age_Per_Job
  Credit_Per_Age
0    2.545302    1    2  ...             0.602624     0.848434       -0.292601
1   -1.190808    0    2  ...             0.542633    -0.396936       -0.797225
```

```
预处理完成后数据概览:
<bound method DataFrame.info of          Age  Sex  Job  ...  Credit_Per_Duration  Age_Per_Job
  Credit_Per_Age
0    2.545302    1    2  ...             0.602624     0.848434       -0.292601
1   -1.190808    0    2  ...             0.542633    -0.396936       -0.797225
2    1.182721    1    1  ...             0.563938     0.591360       -0.352031
3    0.831087    1    2  ...             0.933651     0.277029        1.965414
4    1.534354    1    2  ...             2.205319     0.511451        0.369133
..        ...  ...  ...  ...                  ...          ...             ...
995 -0.399632    0    1  ...             0.736681    -0.199816        1.360977
996  0.391544    1    3  ...             0.275069     0.097886        0.529975
997  0.215727    1    2  ...             1.183893     0.071909       -4.051715
998 -1.102900    1    2  ...            -0.288810    -0.367633        0.458133
999 -0.751266    1    2  ...             0.264203    -0.250422       -0.615263

[1000 rows x 20 columns]>
处理后的数据集已保存到german_credit_processed.csv
..        ...  ...  ...  ...                  ...          ...             ...
995 -0.399632    0    1  ...             0.736681    -0.199816        1.360977
996  0.391544    1    3  ...             0.275069     0.097886        0.529975
997  0.215727    1    2  ...             1.183893     0.071909       -4.051715
998 -1.102900    1    2  ...            -0.288810    -0.367633        0.458133
999 -0.751266    1    2  ...             0.264203    -0.250422       -0.615263

[1000 rows x 20 columns]>
处理后的数据集已保存到german_credit_processed.csv
PS D:\Studies\机器学习\src\course5>
```

（3）相关可视化结果



数值型特征的分布



标签分布

2.模型定义部分

（1）源代码：

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from model import LogisticRegression

# 先划分数据集，分为训练集、测试集和验证集，比例为6：2：2
def train_test_val_split(X, y, test_size=0.2, val_size=0.2, random_state=42):
    np.random.seed(random_state)
    m = len(X)
    indices = np.arange(m)
    np.random.shuffle(indices)

    # 分层抽样
    pos_indices = indices[y[indices] == 1]
    neg_indices = indices[y[indices] == 0]

    # 计算各集合大小
    test_pos_size = int(len(pos_indices) * test_size)
    test_neg_size = int(len(neg_indices) * test_size)

    # 计算验证集大小（基于剩余样本）
    remaining_pos = len(pos_indices) - test_pos_size
    remaining_neg = len(neg_indices) - test_neg_size
    val_pos_size = int(remaining_pos * val_size / (1 - test_size))
    val_neg_size = int(remaining_neg * val_size / (1 - test_size))

    # 划分测试集
    test_pos = pos_indices[:test_pos_size]
    test_neg = neg_indices[:test_neg_size]
    test_indices = np.concatenate([test_pos, test_neg])

    # 划分验证集
    val_pos = pos_indices[test_pos_size:test_pos_size + val_pos_size]
    val_neg = neg_indices[test_neg_size:test_neg_size + val_neg_size]
    val_indices = np.concatenate([val_pos, val_neg])

    # 划分训练集
    train_pos = pos_indices[test_pos_size + val_pos_size:]
    train_neg = neg_indices[test_neg_size + val_neg_size:]
    train_indices = np.concatenate([train_pos, train_neg])

    return (X[train_indices], X[val_indices], X[test_indices],
            y[train_indices], y[val_indices], y[test_indices])

def load_and_train(data_path="german_credit_processed.csv"):
    # 读取数据
    df = pd.read_csv(data_path)
    print("数据形状：", df.shape)

    # 拆分特征与标签
    X = df.drop(columns=["Risk"]).values
    y = df["Risk"].values
    feature_names = df.drop(columns=["Risk"]).columns.tolist()

    # 划分训练集、验证集、测试集
    X_train, X_val, X_test, y_train, y_val, y_test = train_test_val_split(X, y, test_size=0.2, val_size=0.2,
random_state=42)

    # 开始训练模型
    lr_model = LogisticRegression(
        learning_rate=0.1,
```

```
        n_iter=10000,
        early_stopping=True,
        patience=5,
        min_delta=1e-5
    )

    # 传入训练集和验证集
    lr_model.fit(X_train, y_train, X_val, y_val)

    # 打印结果
    print("\n模型拟合结果")
    print(f"\n偏置：{lr_model.b:.4f}")
    print("\n各特征权重系数：")
    for feat, coef in zip(feature_names, lr_model.theta):
        print(f"\n{feat}:{coef:.4f}")


if __name__ == "__main__":
    load_and_train()
```

3.训练方法部分

（1）源代码

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from model import LogisticRegression

# 先划分数据集，分为训练集、测试集和验证集，比例为6：2：2
def train_test_val_split(X, y, test_size=0.2, val_size=0.2, random_state=42):
    np.random.seed(random_state)
    m = len(X)
    indices = np.arange(m)
    np.random.shuffle(indices)

    # 分层抽样
    pos_indices = indices[y[indices] == 1]
    neg_indices = indices[y[indices] == 0]

    # 计算各集合大小
    test_pos_size = int(len(pos_indices) * test_size)
    test_neg_size = int(len(neg_indices) * test_size)

    # 计算验证集大小（基于剩余样本）
    remaining_pos = len(pos_indices) - test_pos_size
    remaining_neg = len(neg_indices) - test_neg_size
    val_pos_size = int(remaining_pos * val_size / (1 - test_size))
    val_neg_size = int(remaining_neg * val_size / (1 - test_size))

    # 划分测试集
    test_pos = pos_indices[:test_pos_size]
    test_neg = neg_indices[:test_neg_size]
    test_indices = np.concatenate([test_pos, test_neg])

    # 划分验证集
    val_pos = pos_indices[test_pos_size:test_pos_size + val_pos_size]
    val_neg = neg_indices[test_neg_size:test_neg_size + val_neg_size]
    val_indices = np.concatenate([val_pos, val_neg])

    # 划分训练集
    train_pos = pos_indices[test_pos_size + val_pos_size:]
    train_neg = neg_indices[test_neg_size + val_neg_size:]
    train_indices = np.concatenate([train_pos, train_neg])
```

```
    return (X[train_indices], X[val_indices], X[test_indices],
            y[train_indices], y[val_indices], y[test_indices])

def load_and_train(data_path="german_credit_processed.csv"):
    # 读取数据
    df = pd.read_csv(data_path)
    print("数据形状：", df.shape)

    # 拆分特征与标签
    X = df.drop(columns=["Risk"]).values
    y = df["Risk"].values
    feature_names = df.drop(columns=["Risk"]).columns.tolist()

    # 划分训练集、验证集、测试集
    X_train, X_val, X_test, y_train, y_val, y_test = train_test_val_split(X, y, test_size=0.2, val_size=0.2,
random_state=42)

    # 开始训练模型
    lr_model = LogisticRegression(
        learning_rate=0.1,
        n_iter=10000,
        early_stopping=True,
        patience=5,
        min_delta=1e-5
    )

    # 传入训练集和验证集
    lr_model.fit(X_train, y_train, X_val, y_val)

    # 打印结果
    print("\n模型拟合结果")
    print(f"\n偏置：{lr_model.b:.4f}")
    print("\n各特征权重系数：")
    for feat, coef in zip(feature_names, lr_model.theta):
        print(f"\n{feat}:{coef:.4f}")


if __name__ == "__main__":
    load_and_train()
```

4.模型性能评估部分

（1）源代码：

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 设置中文字体支持
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

class ModelEvaluator:
    def __init__(self, model, X_test, y_test, feature_names=None):
        self.model = model
        self.X_test = X_test
        self.y_test = y_test
        self.feature_names = feature_names
        self.y_pred = None
        self.y_pred_proba = None

    def evaluate(self, threshold=0.5):
        self.y_pred_proba = self.model.predict_proba(self.X_test)
        self.y_pred = (self.y_pred_proba >= threshold).astype(int)
```

```
        metrics = self._calculate_metrics()
        self._print_metrics(metrics)

        return metrics

    def _calculate_metrics(self):
        # 计算混淆矩阵
        tp = np.sum((self.y_test == 1) & (self.y_pred == 1))
        fp = np.sum((self.y_test == 0) & (self.y_pred == 1))
        tn = np.sum((self.y_test == 0) & (self.y_pred == 0))
        fn = np.sum((self.y_test == 1) & (self.y_pred == 0))

        # 计算指标
        accuracy = (tp + tn) / len(self.y_test) if len(self.y_test) > 0 else 0
        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0
        f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

        # 手动计算AUC
        auc = self._manual_auc()

        return {
            'accuracy': accuracy,
            'precision': precision,
            'recall': recall,
            'f1_score': f1,
            'auc': auc,
            'confusion_matrix': np.array([[tn, fp], [fn, tp]])
        }

    def _manual_auc(self):
        # 对预测概率进行排序
        sorted_indices = np.argsort(self.y_pred_proba)[::-1]
        sorted_labels = self.y_test[sorted_indices]

        # 计算真正率和假正率
        total_pos = np.sum(self.y_test == 1)
        total_neg = np.sum(self.y_test == 0)

        if total_pos == 0 or total_neg == 0:
            return 0.5

        tpr = [0]
        fpr = [0]
        tp_count = 0
        fp_count = 0

        for label in sorted_labels:
            if label == 1:
                tp_count += 1
            else:
                fp_count += 1
            tpr.append(tp_count / total_pos)
            fpr.append(fp_count / total_neg)

        # 计算AUC（梯形法则）
        auc = 0
        for i in range(1, len(fpr)):
            auc += (fpr[i] - fpr[i-1]) * (tpr[i] + tpr[i-1]) / 2

        return auc

    def _print_metrics(self, metrics):
        print("模型性能评估结果")
```

```
        print(f"准确率 (Accuracy): {metrics['accuracy']:.4f}")
        print(f"精确率 (Precision): {metrics['precision']:.4f}")
        print(f"召回率 (Recall): {metrics['recall']:.4f}")
        print(f"F1分数 (F1-Score): {metrics['f1_score']:.4f}")
        print(f"AUC值: {metrics['auc']:.4f}")

        # 手动生成分类报告
        cm = metrics['confusion_matrix']
        print(f"\n混淆矩阵:")
        print(f"真负例 (TN): {cm[0, 0]}, 假正例 (FP): {cm[0, 1]}")
        print(f"假负例 (FN): {cm[1, 0]}, 真正例 (TP): {cm[1, 1]}")

        # 计算各类别的指标
        support_0 = cm[0, 0] + cm[0, 1]  # 负例样本数
        support_1 = cm[1, 0] + cm[1, 1]  # 正例样本数

        precision_0 = cm[0, 0] / (cm[0, 0] + cm[1, 0]) if (cm[0, 0] + cm[1, 0]) > 0 else 0
        recall_0 = cm[0, 0] / support_0 if support_0 > 0 else 0
        f1_0 = 2 * precision_0 * recall_0 / (precision_0 + recall_0) if (precision_0 + recall_0) > 0 else 0

        precision_1 = metrics['precision']
        recall_1 = metrics['recall']
        f1_1 = metrics['f1_score']

        print("\n分类报告:")
        print("              Precision    Recall    F1-Score    Support")
        print(f"Good (0)       {precision_0:.4f}      {recall_0:.4f}     {f1_0:.4f}       {support_0}")
        print(f"Bad (1)        {precision_1:.4f}      {recall_1:.4f}     {f1_1:.4f}       {support_1}")
        print(f"Accuracy                           {metrics['accuracy']:.4f}       {len(self.y_test)}")

    def plot_confusion_matrix(self):
        cm = self._calculate_metrics()['confusion_matrix']

        plt.figure(figsize=(8, 6))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                    xticklabels=['Good', 'Bad'],
                    yticklabels=['Good', 'Bad'])
        plt.title('混淆矩阵')
        plt.xlabel('预测标签')
        plt.ylabel('真实标签')
        plt.tight_layout()
        plt.show()

    def plot_roc_curve(self):
        # 对预测概率进行排序
        sorted_indices = np.argsort(self.y_pred_proba)[::-1]
        sorted_labels = self.y_test[sorted_indices]

        total_pos = np.sum(self.y_test == 1)
        total_neg = np.sum(self.y_test == 0)

        if total_pos == 0 or total_neg == 0:
            print("无法绘制ROC曲线：正例或负例样本数为0")
            return

        tpr = [0]
        fpr = [0]
        tp_count = 0
        fp_count = 0

        for label in sorted_labels:
            if label == 1:
                tp_count += 1
```

```
            else:
                fp_count += 1
            tpr.append(tp_count / total_pos)
            fpr.append(fp_count / total_neg)

        # 计算AUC
        auc = 0
        for i in range(1, len(fpr)):
            auc += (fpr[i] - fpr[i-1]) * (tpr[i] + tpr[i-1]) / 2

        plt.figure(figsize=(8, 6))
        plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC曲线 (AUC = {auc:.4f})')
        plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='随机分类器')
        plt.xlim([0.0, 1.0])
        plt.ylim([0.0, 1.05])
        plt.xlabel('假正率 (False Positive Rate)')
        plt.ylabel('真正率 (True Positive Rate)')
        plt.title('ROC曲线')
        plt.legend(loc="lower right")
        plt.grid(True)
        plt.tight_layout()
        plt.show()

    def plot_feature_importance(self):
        if self.feature_names is None:
            print("未提供特征名称，无法绘制特征重要性图")
            return

        importance = np.abs(self.model.theta)
        feature_importance_df = pd.DataFrame({
            'feature': self.feature_names,
            'importance': importance
        }).sort_values('importance', ascending=True)

        plt.figure(figsize=(10, 8))
        plt.barh(feature_importance_df['feature'], feature_importance_df['importance'])
        plt.xlabel('特征重要性 (绝对值)')
        plt.title('特征重要性排序')
        plt.tight_layout()
        plt.show()

    def plot_loss_history(self):
        if not hasattr(self.model, 'train_loss_history') or not self.model.train_loss_history:
            print("模型没有训练损失历史记录")
            return

        plt.figure(figsize=(10, 6))

        epochs = range(1, len(self.model.train_loss_history) + 1)
        plt.plot(epochs, self.model.train_loss_history, label='训练损失', color='blue')

        if hasattr(self.model, 'val_loss_history') and self.model.val_loss_history:
            plt.plot(epochs, self.model.val_loss_history, label='验证损失', color='red')

        plt.xlabel('迭代次数')
        plt.ylabel('损失值')
        plt.title('训练过程损失变化')
        plt.legend()
        plt.grid(True)
        plt.tight_layout()
        plt.show()

    def plot_threshold_analysis(self):
```

```
        thresholds = np.linspace(0.1, 0.9, 9)
        accuracies = []
        precisions = []
        recalls = []
        f1_scores = []

        for threshold in thresholds:
            y_pred_thresh = (self.y_pred_proba >= threshold).astype(int)

            # 手动计算指标
            tp = np.sum((self.y_test == 1) & (y_pred_thresh == 1))
            fp = np.sum((self.y_test == 0) & (y_pred_thresh == 1))
            tn = np.sum((self.y_test == 0) & (y_pred_thresh == 0))
            fn = np.sum((self.y_test == 1) & (y_pred_thresh == 0))

            accuracy = (tp + tn) / len(self.y_test) if len(self.y_test) > 0 else 0
            precision = tp / (tp + fp) if (tp + fp) > 0 else 0
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0
            f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

            accuracies.append(accuracy)
            precisions.append(precision)
            recalls.append(recall)
            f1_scores.append(f1)

        plt.figure(figsize=(12, 8))
        plt.plot(thresholds, accuracies, label='准确率', marker='o')
        plt.plot(thresholds, precisions, label='精确率', marker='s')
        plt.plot(thresholds, recalls, label='召回率', marker='^')
        plt.plot(thresholds, f1_scores, label='F1分数', marker='d')

        plt.xlabel('分类阈值')
        plt.ylabel('性能指标')
        plt.title('不同阈值下的模型性能')
        plt.legend()
        plt.grid(True)
        plt.tight_layout()
        plt.show()

def evaluate_model(model, X_test, y_test, feature_names=None, plot_all=True):
    evaluator = ModelEvaluator(model, X_test, y_test, feature_names)

    metrics = evaluator.evaluate()

    if plot_all:
        evaluator.plot_confusion_matrix()
        evaluator.plot_roc_curve()
        evaluator.plot_feature_importance()
        evaluator.plot_loss_history()
        evaluator.plot_threshold_analysis()

    return metrics

if __name__ == "__main__":
    print("这是模型评估模块，请通过main.py或导入使用")
```

5.总入口

（1）源代码：

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from model import LogisticRegression
from train import train_test_val_split
from evaluate import evaluate_model
```

```
def load_data(data_path="german_credit_processed.csv"):
    df = pd.read_csv(data_path)
    print(f"数据形状：{df.shape}")

    X = df.drop(columns=["Risk"]).values
    y = df["Risk"].values
    feature_names = df.drop(columns=["Risk"]).columns.tolist()

    return X, y, feature_names

def train_model(X, y, feature_names):
    print("\n正在划分数据集...")
    X_train, X_val, X_test, y_train, y_val, y_test = train_test_val_split(
        X, y, test_size=0.2, val_size=0.2, random_state=42
    )

    print(f"训练集大小：{X_train.shape[0]}")
    print(f"验证集大小：{X_val.shape[0]}")
    print(f"测试集大小：{X_test.shape[0]}")

    print("\n正在训练逻辑回归模型...")
    model = LogisticRegression(
        learning_rate=0.1,
        n_iter=10000,
        early_stopping=True,
        patience=5,
        min_delta=1e-5
    )

    model.fit(X_train, y_train, X_val, y_val)

    print("\n模型训练完成！")
    print(f"偏置：{model.b:.4f}")
    print("\n各特征权重系数：")
    for feat, coef in zip(feature_names, model.theta):
        print(f"{feat}: {coef:.4f}")

    return model, X_test, y_test

def evaluate_and_visualize(model, X_test, y_test, feature_names):
    metrics = evaluate_model(model, X_test, y_test, feature_names, plot_all=True)

    return metrics

def main():
    # 1. 加载数据
    X, y, feature_names = load_data()

    # 2. 训练模型
    model, X_test, y_test = train_model(X, y, feature_names)

    # 3. 评估和可视化
    metrics = evaluate_and_visualize(model, X_test, y_test, feature_names)


    # 打印最终性能摘要
    print("\n最终模型性能摘要：")
    print(f"测试集准确率：{metrics['accuracy']:.4f}")
    print(f"AUC值：{metrics['auc']:.4f}")
    print(f"F1分数：{metrics['f1_score']:.4f}")

if __name__ == "__main__":
    main()
```

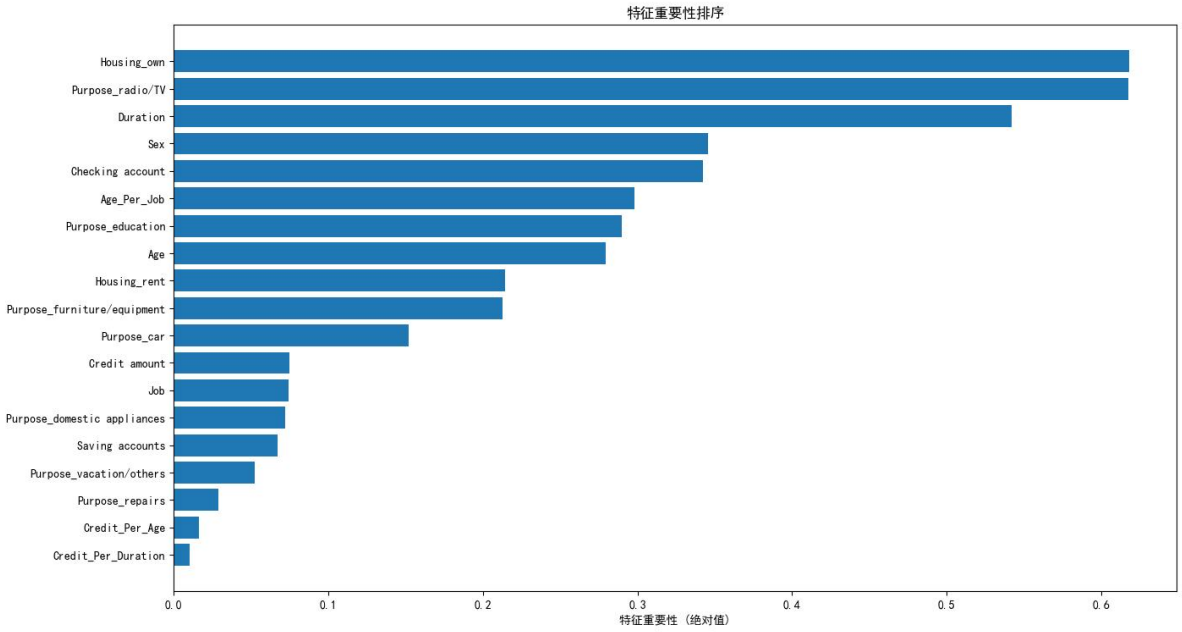
6. 整体结果分析：

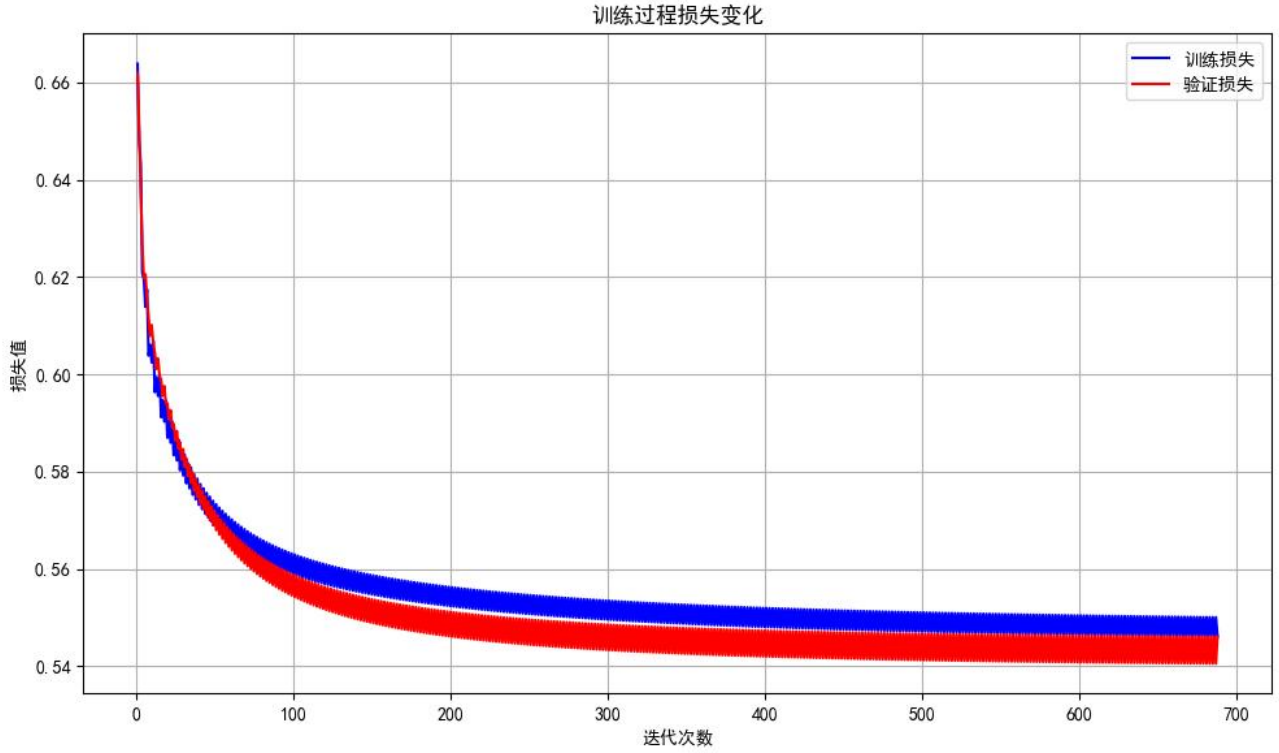
（1）运行结果：

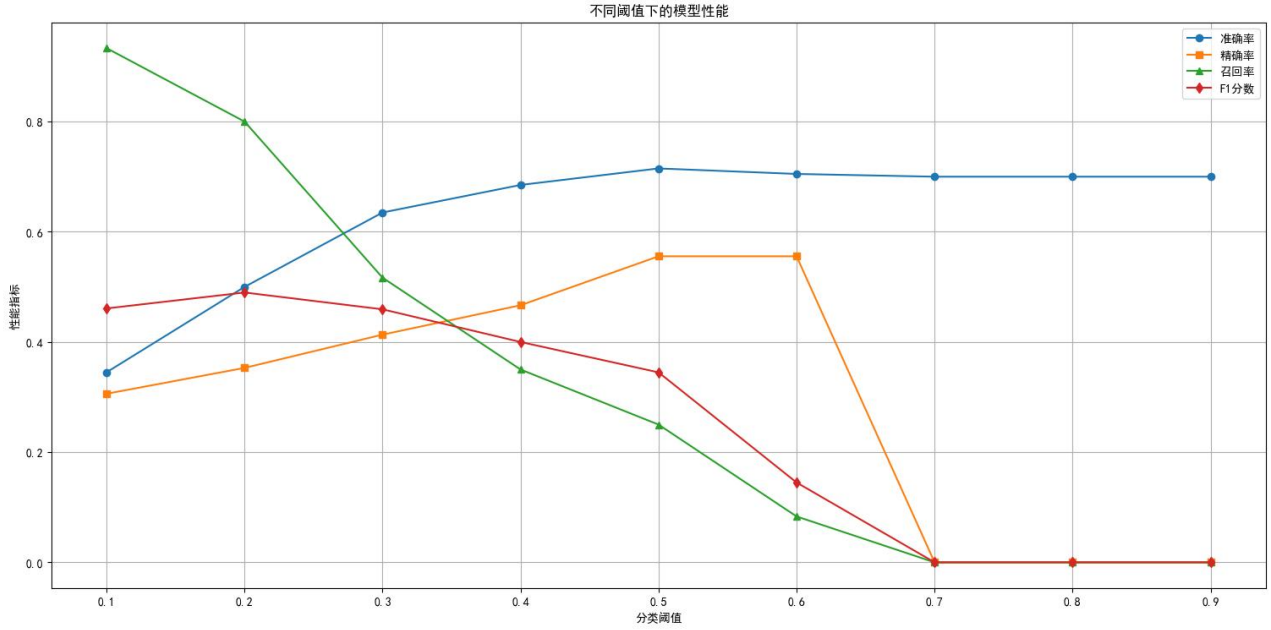
```
数据形状：(1000, 20)

正在划分数据集...
训练集大小：600
验证集大小：200
测试集大小：200

正在训练逻辑回归模型...

早停触发，共迭代688次，最佳损失：0.5405

模型训练完成！
偏置：-0.2651

各特征权重系数：
Age: -0.2794
Sex: -0.3457
Job: -0.0744
Saving accounts: -0.0670
Checking account: 0.3425
Credit amount: -0.0749
Duration: 0.5420
Housing_own: -0.6179
Housing_rent: -0.2146
Purpose_car: 0.1521
Purpose_domestic appliances: -0.0722
Purpose_education: 0.2898
Purpose_furniture/equipment: -0.2126
Purpose_radio/TV: -0.6174
Purpose_repairs: -0.0287
Purpose_vacation/others: 0.0526
Credit_Per_Duration: -0.0101
Age_Per_Job: -0.2979
Credit_Per_Age: 0.0160
模型性能评估结果
准确率 (Accuracy): 0.7150
精确率 (Precision): 0.5556
召回率 (Recall): 0.2500
```

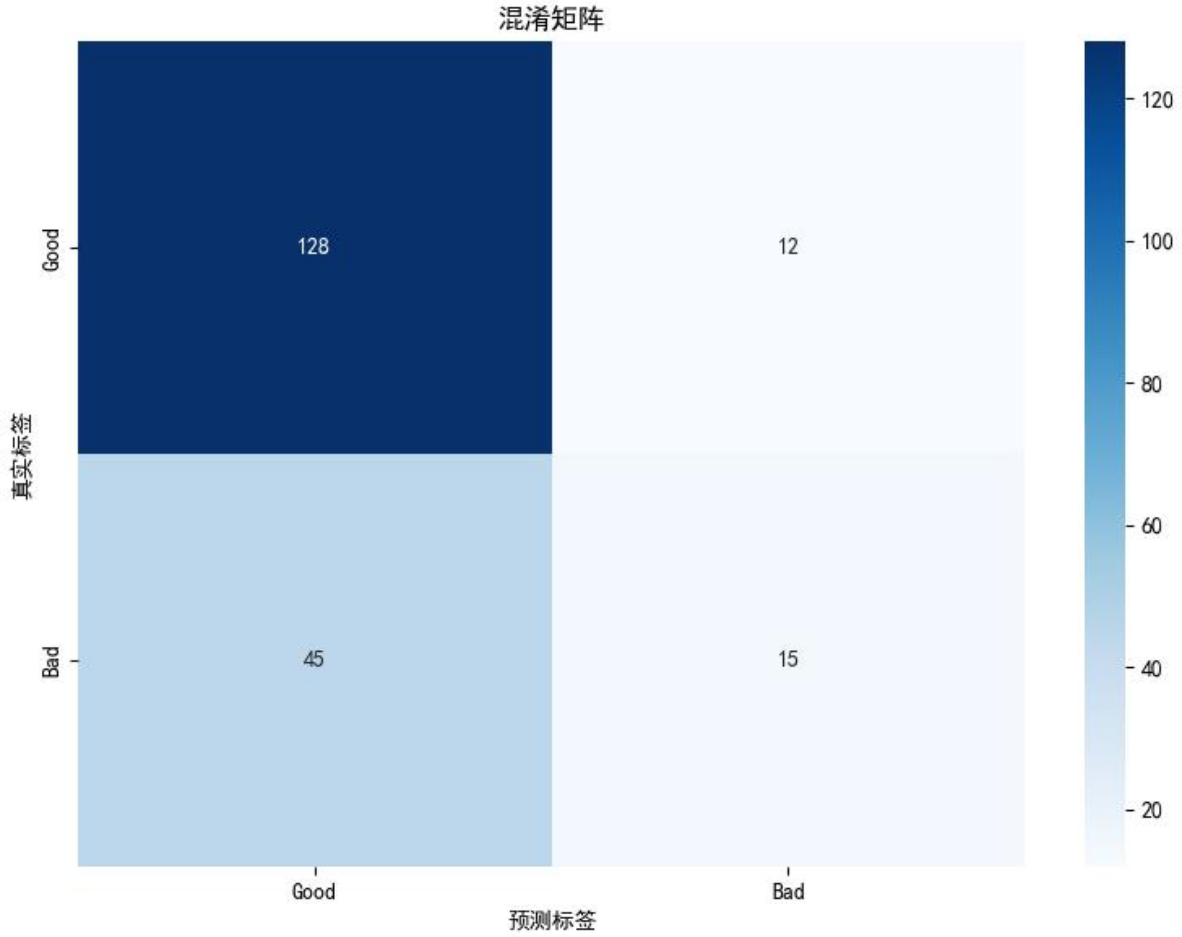
```
模型性能评估结果
准确率 (Accuracy): 0.7150
精确率 (Precision): 0.5556
召回率 (Recall): 0.2500
F1分数 (F1-Score): 0.3448
AUC值: 0.6389

混淆矩阵:
真负例 (TN): 128, 假正例 (FP): 12
假负例 (FN): 45, 真正例 (TP): 15

分类报告:
              Precision    Recall    F1-Score    Support
Good (0)         0.7399    0.9143      0.8179        140
Bad (1)          0.5556    0.2500      0.3448         60
Accuracy                             0.7150        200

最终模型性能摘要:
测试集准确率: 0.7150
AUC值: 0.6389
F1分数: 0.3448
PS D:\Studies\机器学习\src\course5>
```

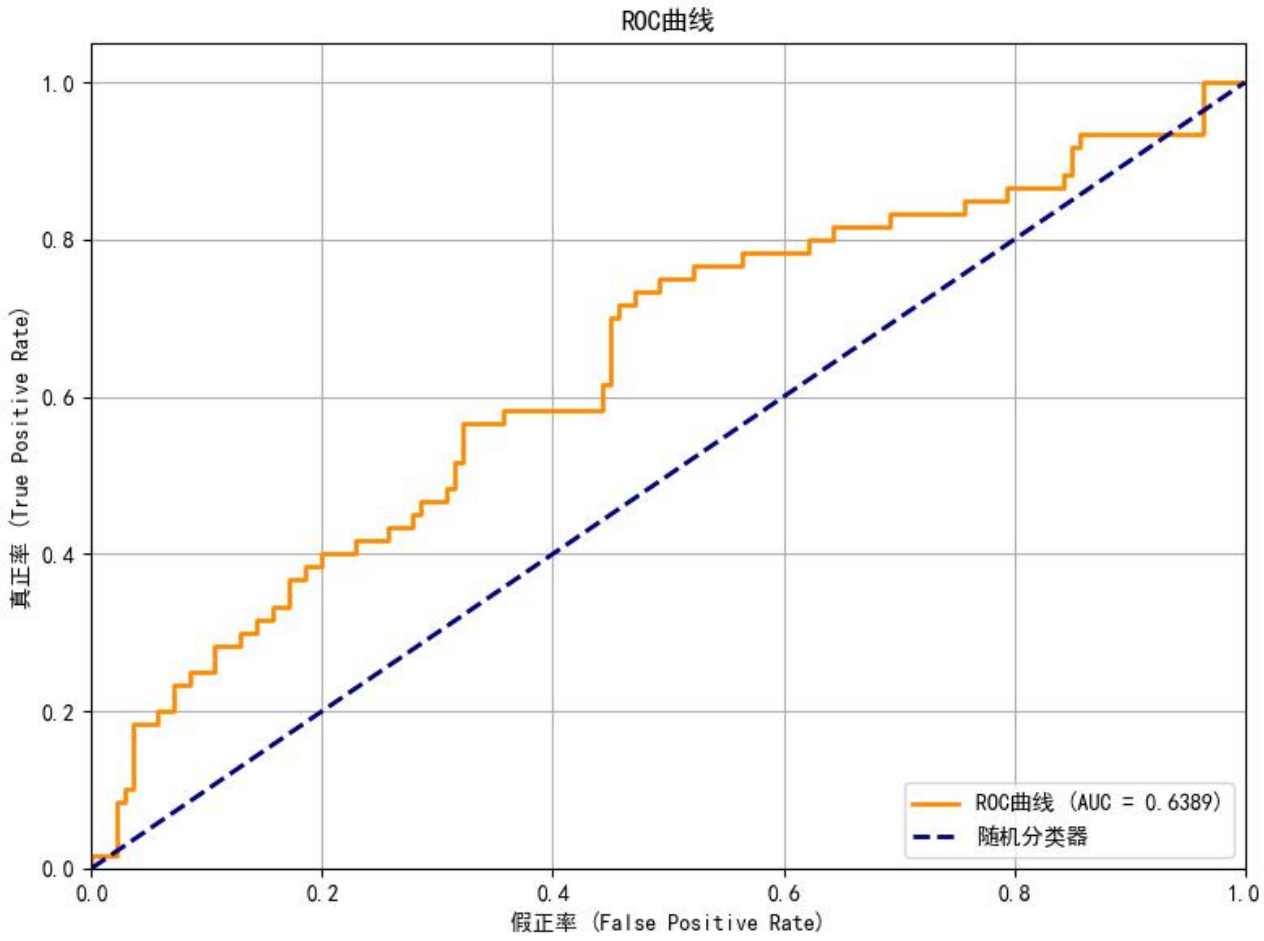
（2）可视化结果




训练过程损失变化
训练损失
验证损失
损失值
迭代次数
0.66
0.64
0.62
0.60
0.58
0.56
0.54
0
100
200
300
400
500
600
700


不同阈值下的模型性能
准确率
精确率
召回率
F1分数
性能指标
分类阈值
0.8
0.6
0.4
0.2
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9


混淆矩阵
128
12
45
15
Good
Bad
真实标签
Good
Bad
预测标签
120
100
80
60
40
20


ROC曲线
真正率 (True Positive Rate)
假正率 (False Positive Rate)
0.0
0.2
0.4
0.6
0.8
1.0
ROC曲线 (AUC = 0.6389)
随机分类器

6.分类决策边界

（1）源代码

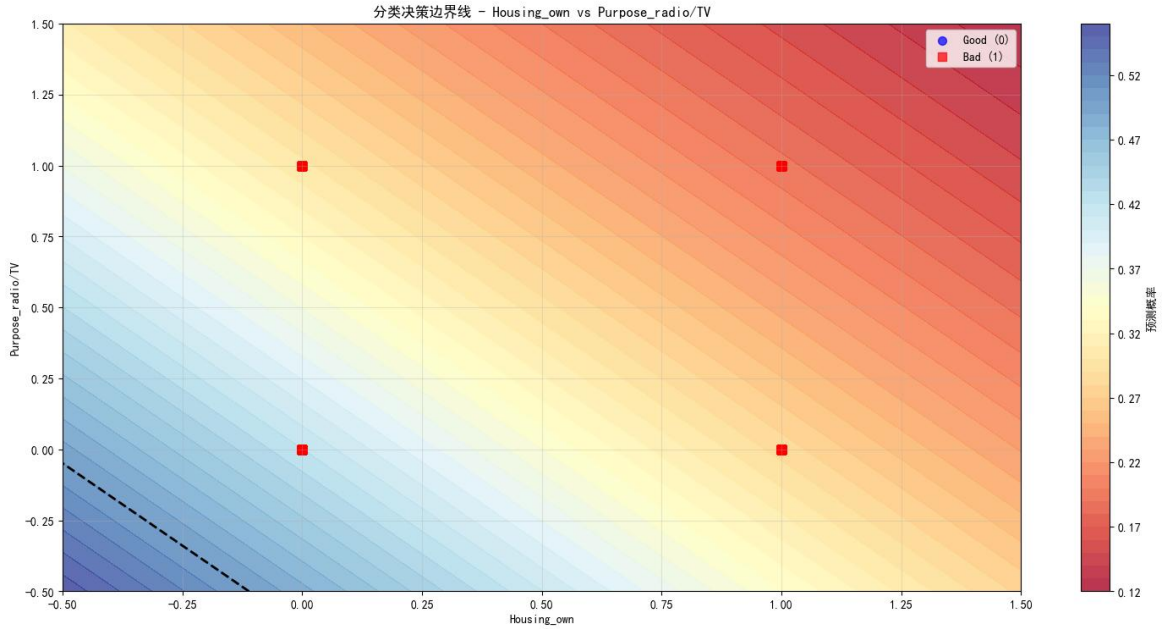
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from model import LogisticRegression
from train import train_test_val_split

# 设置中文字体支持
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

def select_top_features(data_path="german_credit_processed.csv", top_n=2):
    df = pd.read_csv(data_path)

    # 加载数据
    X = df.drop(columns=["Risk"]).values
    y = df["Risk"].values
    feature_names = df.drop(columns=["Risk"]).columns.tolist()

    # 训练模型获取特征权重
    X_train, X_val, X_test, y_train, y_val, y_test = train_test_val_split(
        X, y, test_size=0.2, val_size=0.2, random_state=42
    )

    model = LogisticRegression(
        learning_rate=0.1,
        n_iter=10000,
        early_stopping=True,
        patience=5,
        min_delta=1e-5
    )

    model.fit(X_train, y_train, X_val, y_val)

    # 根据权重绝对值选择最重要的特征
    feature_importance = np.abs(model.theta)
    top_indices = np.argsort(feature_importance)[-top_n:][::-1]

    top_features = [feature_names[i] for i in top_indices]
    top_importance = [feature_importance[i] for i in top_indices]

    print("最相关的两个特征：")
    for i, (feat, imp) in enumerate(zip(top_features, top_importance)):
        print(f"{i+1}. {feat}: {imp:.4f}")

    return top_features, top_indices, X, y, feature_names

def plot_decision_boundary(data_path="german_credit_processed.csv"):

    # 选择最相关的两个特征
    top_features, top_indices, X, y, feature_names = select_top_features(data_path)

    # 提取这两个特征的数据
    X_top = X[:, top_indices]

    # 创建网格数据
    x_min, x_max = X_top[:, 0].min() - 0.5, X_top[:, 0].max() + 0.5
    y_min, y_max = X_top[:, 1].min() - 0.5, X_top[:, 1].max() + 0.5

    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                         np.linspace(y_min, y_max, 100))
```

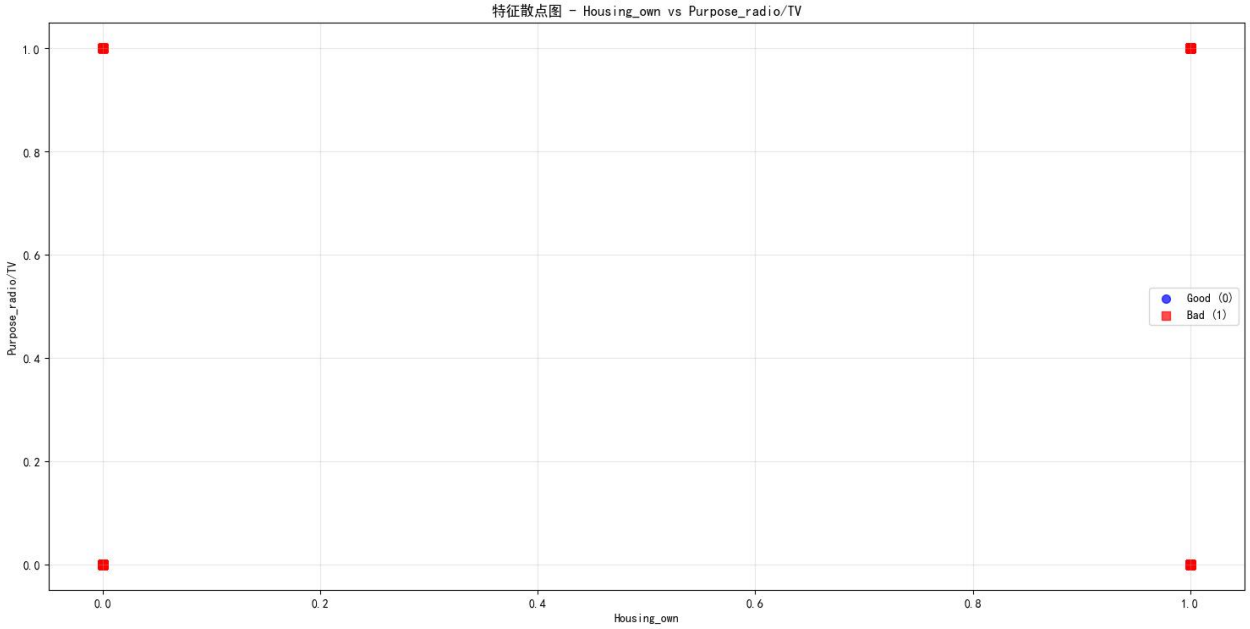
```
    # 只使用这两个特征训练新模型
    X_train, X_val, X_test, y_train, y_val, y_test = train_test_val_split(
        X_top, y, test_size=0.2, val_size=0.2, random_state=42
    )

    model_2d = LogisticRegression(
        learning_rate=0.1,
        n_iter=5000,
        early_stopping=True,
        patience=5,
        min_delta=1e-5
    )

    model_2d.fit(X_train, y_train, X_val, y_val)

    # 预测网格点
    Z = model_2d.predict_proba(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # 绘制决策边界
    plt.figure(figsize=(12, 8))

    # 绘制等高线（决策边界）
    contour = plt.contourf(xx, yy, Z, levels=50, alpha=0.8, cmap='RdYlBu')
    plt.colorbar(contour, label='预测概率')

    # 绘制决策边界线
    plt.contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=2, linestyles='dashed')

    # 绘制数据点
    scatter = plt.scatter(X_top[y == 0, 0], X_top[y == 0, 1],
                          c='blue', marker='o', label='Good (0)', alpha=0.7, s=50)
    plt.scatter(X_top[y == 1, 0], X_top[y == 1, 1],
                c='red', marker='s', label='Bad (1)', alpha=0.7, s=50)

    plt.xlabel(top_features[0])
    plt.ylabel(top_features[1])
    plt.title(f'分类决策边界线 - {top_features[0]} vs {top_features[1]}')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

    # 打印模型信息
    print(f"\n二维模型参数：")
    print(f"偏置: {model_2d.b:.4f}")
    print(f"{top_features[0]}权重: {model_2d.theta[0]:.4f}")
    print(f"{top_features[1]}权重: {model_2d.theta[1]:.4f}")

    # 计算二维模型的性能
    y_pred_2d = model_2d.predict(X_test)
    accuracy_2d = np.mean(y_pred_2d == y_test)
    print(f"二维模型测试集准确率: {accuracy_2d:.4f}")

def plot_feature_scatter(data_path="german_credit_processed.csv"):

    # 选择最相关的两个特征
    top_features, top_indices, X, y, feature_names = select_top_features(data_path)

    # 提取这两个特征的数据
    X_top = X[:, top_indices]

    # 绘制散点图
```

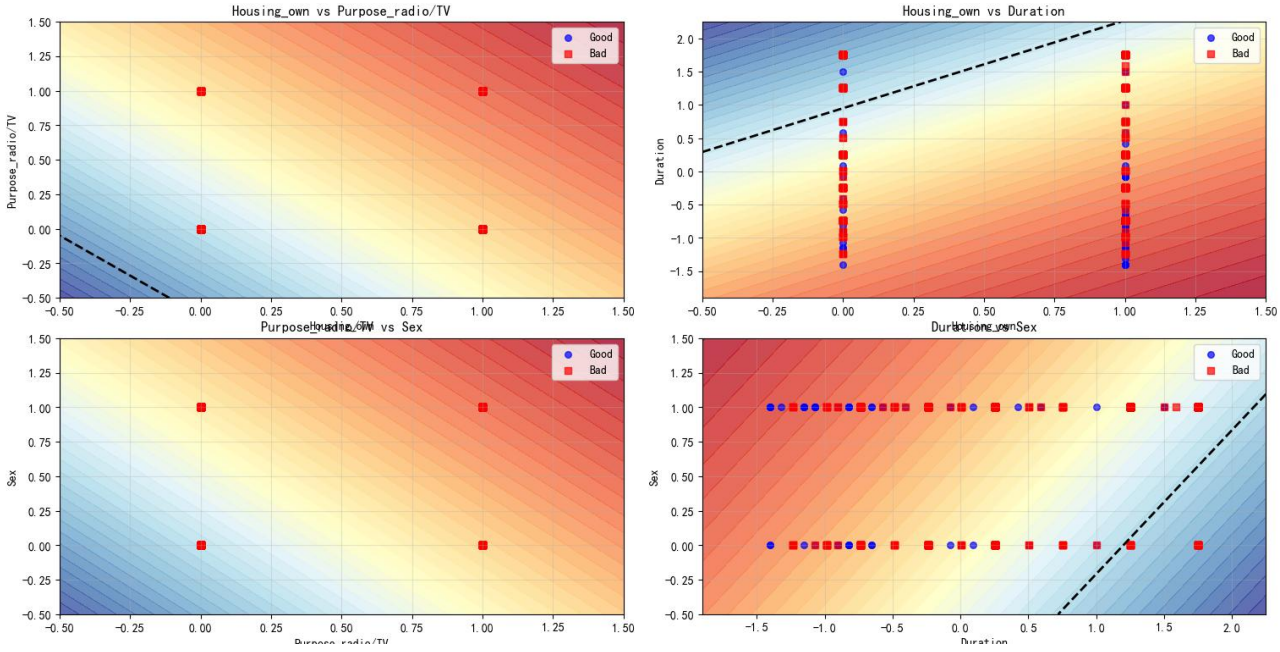
```
    plt.figure(figsize=(10, 8))

    plt.scatter(X_top[y == 0, 0], X_top[y == 0, 1],
                c='blue', marker='o', label='Good (0)', alpha=0.7, s=50)
    plt.scatter(X_top[y == 1, 0], X_top[y == 1, 1],
                c='red', marker='s', label='Bad (1)', alpha=0.7, s=50)

    plt.xlabel(top_features[0])
    plt.ylabel(top_features[1])
    plt.title(f'特征散点图 - {top_features[0]} vs {top_features[1]}')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

def plot_multiple_decision_boundaries(data_path="german_credit_processed.csv"):

    df = pd.read_csv(data_path)
    X = df.drop(columns=["Risk"]).values
    y = df["Risk"].values
    feature_names = df.drop(columns=["Risk"]).columns.tolist()

    # 训练完整模型获取特征重要性
    X_train, X_val, X_test, y_train, y_val, y_test = train_test_val_split(
        X, y, test_size=0.2, val_size=0.2, random_state=42
    )

    model = LogisticRegression(
        learning_rate=0.1,
        n_iter=10000,
        early_stopping=True,
        patience=5,
        min_delta=1e-5
    )

    model.fit(X_train, y_train, X_val, y_val)

    # 选择前4个最重要的特征
    feature_importance = np.abs(model.theta)
    top_indices = np.argsort(feature_importance)[-4:][::-1]
    top_features = [feature_names[i] for i in top_indices]

    # 创建多个子图
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))
    axes = axes.ravel()

    # 绘制不同的特征组合
    combinations = [(0, 1), (0, 2), (1, 3), (2, 3)]

    for idx, (i, j) in enumerate(combinations):
        ax = axes[idx]

        # 提取特征数据
        X_comb = X[:, [top_indices[i], top_indices[j]]]

        # 创建网格
        x_min, x_max = X_comb[:, 0].min() - 0.5, X_comb[:, 0].max() + 0.5
        y_min, y_max = X_comb[:, 1].min() - 0.5, X_comb[:, 1].max() + 0.5

        xx, yy = np.meshgrid(np.linspace(x_min, x_max, 50),
                             np.linspace(y_min, y_max, 50))

        # 训练二维模型
        X_train_2d, X_val_2d, X_test_2d, y_train_2d, y_val_2d, y_test_2d = train_test_val_split(
```

```
            X_comb, y, test_size=0.2, val_size=0.2, random_state=42
        )

        model_2d = LogisticRegression(
            learning_rate=0.1,
            n_iter=3000,
            early_stopping=True,
            patience=5,
            min_delta=1e-5
        )

        model_2d.fit(X_train_2d, y_train_2d, X_val_2d, y_val_2d)

        # 预测网格点
        Z = model_2d.predict_proba(np.c_[xx.ravel(), yy.ravel()])
        Z = Z.reshape(xx.shape)

        # 绘制决策边界
        contour = ax.contourf(xx, yy, Z, levels=50, alpha=0.8, cmap='RdYlBu')
        ax.contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=2, linestyles='dashed')

        # 绘制数据点
        ax.scatter(X_comb[y == 0, 0], X_comb[y == 0, 1],
                   c='blue', marker='o', label='Good', alpha=0.7, s=30)
        ax.scatter(X_comb[y == 1, 0], X_comb[y == 1, 1],
                   c='red', marker='s', label='Bad', alpha=0.7, s=30)

        ax.set_xlabel(top_features[i])
        ax.set_ylabel(top_features[j])
        ax.set_title(f'{top_features[i]} vs {top_features[j]}')
        ax.legend()
        ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

if __name__ == "__main__":

    # 绘制主要决策边界
    plot_decision_boundary()

    # 绘制特征散点图
    plot_feature_scatter()

    # 绘制多个决策边界
    plot_multiple_decision_boundaries()
```

（2）运行结果：


分类决策边界线 - Housing_own vs Purpose_radio/TV
Good (0)
Bad (1)
Purpose_radio/TV
Housing_own
预测概率


特征散点图 - Housing_own vs Purpose_radio/TV
Good (0)
Bad (1)
Purpose_radio/TV
Housing_own


Housing_own vs Purpose_radio/TV
Housing_own vs Duration
Purpose_radio/TV vs Sex
Duration vs Sex
Good
Bad
Purpose_radio/TV
Duration
Sex

4 实验总结与讨论

1. 总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题。

2. 讨论最终得到的结果是否合理、是否达到题目要求； 分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法。

本次实验围绕德国信用风险分类任务，完整实现了从数据预处理到模型训练、评估、可视化的全流程，成功达成“掌握混合型数据处理方式、加深逻辑回归模型理解”的核心目标。通过实践，明确了不同问题场景的适配方法：对于含数值型与类别型的混合型数据，需先通过缺失值填充（将储蓄账户、支票账户的缺失值设为“none”独立类别）、特征编码（二分类特征标签编码、有序多分类特征整数编码、无序多分类特征独热编码）、数值标准化（Z-Score）及异常值截断（IQR 法）完成数据清洗；对于二分类预测任务，逻辑回归模型通过 Sigmoid 函数映射概率、梯度下降优化参数，结合早停机制可有效避免过拟合，提升模型泛化能力；模型评估需兼顾准确率、精确率、召回率等多维度指标，配合特征重要性、决策边界等可视化手段，能更全面地分析模型性能。

实验中深入掌握了关键技术要点：手动实现了 Sigmoid 激活函数、梯度下降参数更新及早停逻辑，理解了逻辑回归“线性预测 + 概率映射”的本质；掌握了分层抽样划分训练集、验证集、测试集的方法，确保数据分布一致性；学会了手动计算 AUC、F1 分数等评估指标，以及通过特征权重绝对值筛选重要特征的技巧。这些实践经验为后续处理分类任务提供了可复用的技术框架。

本次实验过程中遇到了三类典型问题，通过针对性调整得以解决：一是代码语法与逻辑错误，如模型训练时梯度计算中误用未定义变量、np.concatenate函数参数嵌套多余方括号，通过逐行排查代码、核对变量定义及函数参数格式，修正了梯度计算的标签引用错误和数组拼接格式问题；二是数据划分不合理，初始划分未考虑类别分布，导致验证集代表性不足，通过分层抽样按比例分配正负例样本，确保了训练集、验证集、测试集的信用等级分布一致；三是模型收敛速度慢，初始迭代次数设置不合理，启用早停机制（耐心值 5、最小损失改善幅度 1e-5）后，模型在 688 次迭代时即达到最优验证损失，既节省了训练时间，又避免了过拟合。此外，针对可视化中中文显示乱码的问题，通过设置支持中文的字体库得以解决。

实验最终取得了测试集准确率 71.5%、AUC 值 0.6389 的结果，整体符合逻辑回归模型的性能预期，达到了题目要求。从结果来看，模型能较好地识别信用良好（Good）的用户（召回率 91.43%），但对信用不良（Bad）用户的识别能力较弱（召回率 25.00%），这与数据集中类别不平衡（Good 用户占比 70%、Bad 用户占比 30%）密切相关，模型天然倾向于预测占比更高的类别。特征重要性分析显示，信贷期限（Duration）、住房类型（Housing_own）、信贷用途（Purpose_radio/TV）及性别（Sex）是影响信用风险的关键因素，其中信贷期限越长、无自有住房的用户信用风险更高，符合信贷业务的实际逻辑，说明模型捕捉到了数据中的核心规律。

模型存在的主要不足是对少数类（Bad）的识别能力不足，导致 F1 分数仅为 0.3448。产生该问题的原因包括：数据类别不平衡未做专门处理、逻辑回归作为线性模型难以捕捉复杂非线性关系、特征工程的衍生特征针对性不足。针对这些问题，可通过过采样少数类、欠采样多数类平衡数据分布，引入多项式特征捕捉非线性关系，或结合业务场景增加更多有效衍生特征（如信贷金额与收入的比值）等方式优化；同时，调整学习率、迭代次数等超参数，或尝试 L1/L2 正则化减少特征冗余，也可能进一步提升模型性能。

本次实验通过理论与实践结合，不仅夯实了逻辑回归的数学原理与实现细节，更培养了“数据清洗→模型设计→训练优化→结果分析”的系统性思维。实验结果验证了逻辑回归在信用风险分类任务中的

有效性，其模型简单、可解释性强的特点，适合作为此类场景的基准模型。未来可进一步探索：尝试随机森林、SVM 等非线性模型，对比其与逻辑回归的性能差异；引入更多业务相关特征（如还款历史、收入水平），提升模型预测精度；针对类别不平衡问题，探索 SMOTE 等更先进的采样方法，改善少数类识别效果。这些延伸方向将有助于更深入地解决信用风险预测中的实际问题，提升模型的实用价值。